

Auffret Gatien
Bergeret Aubin
Cojean Baptiste
ESIR 2 SI



**Université
de Rennes**

Conception Orientée-Objet

Mini éditeur de texte



Sommaire

Sommaire.....	2
Introduction.....	2
Analyse fonctionnelle :.....	3
Diagramme de cas d'utilisation V1 :	3
Diagramme de cas d'utilisation V2 :	4
Analyse structurelle :.....	5
Diagramme de classe V1 :	5
Diagramme de classe V2 :	5
Diagrammes de séquence:.....	6
Séquence "Couper" :	6
Séquence "Insérer du texte" :	6
Workspaces :.....	7
V1 :	7
V2 :	7
Conclusion :.....	9

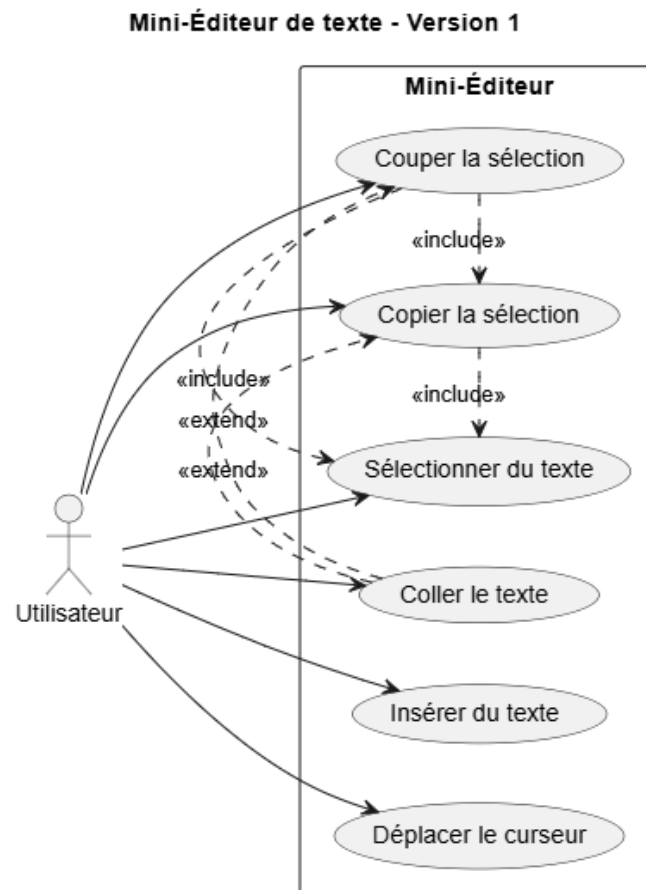
Introduction

Dans ce TP, nous allons concevoir un mini-éditeur de texte en utilisant la programmation orientée objet, notamment avec le langage Java et l'outil GitLab pour la gestion du projet.

L'objectif est de mettre en œuvre plusieurs fonctionnalités dans une zone de travail (le buffer) comme la sélection, la copie, la coupe et le collage de texte. L'éditeur communique avec l'utilisateur via une interface textuelle, et le rapport présente la conception du programme à l'aide d'un diagramme de classes et de diagrammes de séquence permettant de visualiser la structure et le fonctionnement de l'application.

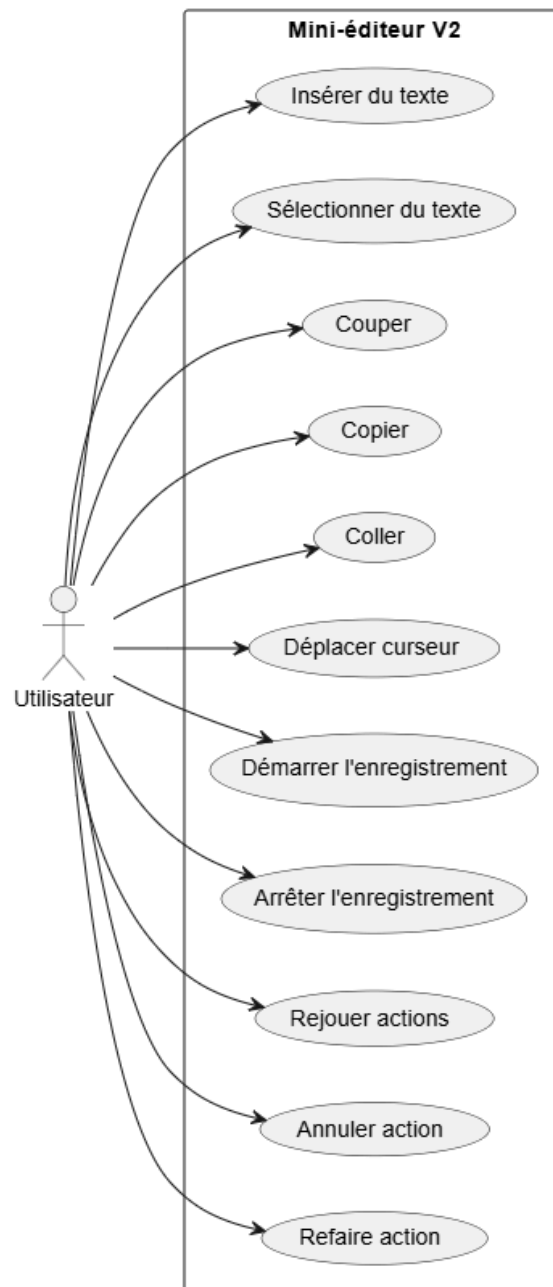
Analyse fonctionnelle :

Diagramme de cas d'utilisation V1 :



L'utilisateur effectue des actions d'édition de base : insérer du texte, sélectionner une portion, couper, copier, coller et déplacer le curseur. Chaque cas d'utilisation correspond à une commande accessible dans l'interface graphique.

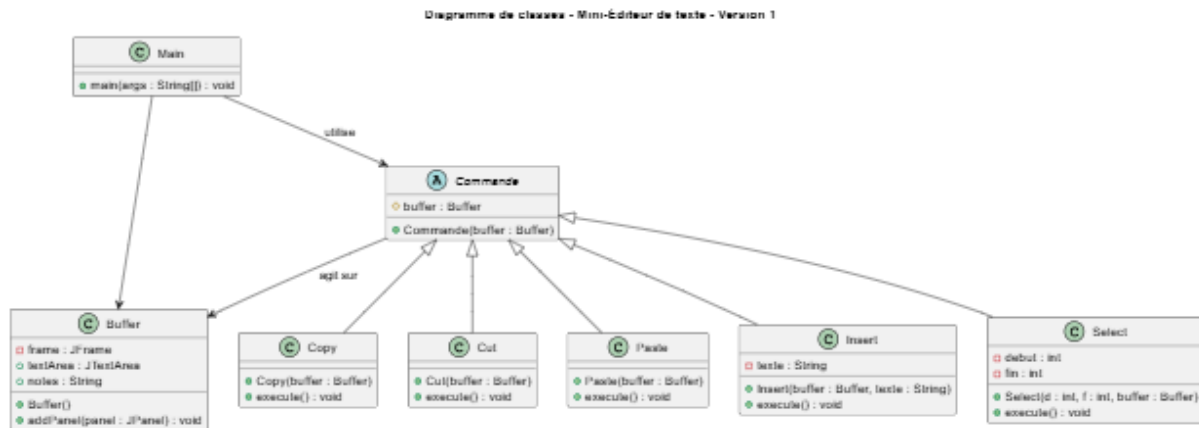
Diagramme de cas d'utilisation V2 :



On ajoute de nouvelles possibilités. L'utilisateur peut démarrer et arrêter l'enregistrement des actions, rejouer une séquence enregistrée, annuler une action et la refaire. Ces nouveaux cas permettent le contrôle de l'historique et de la répétition des commandes.

Analyse structurelle :

Diagramme de classe V1 :

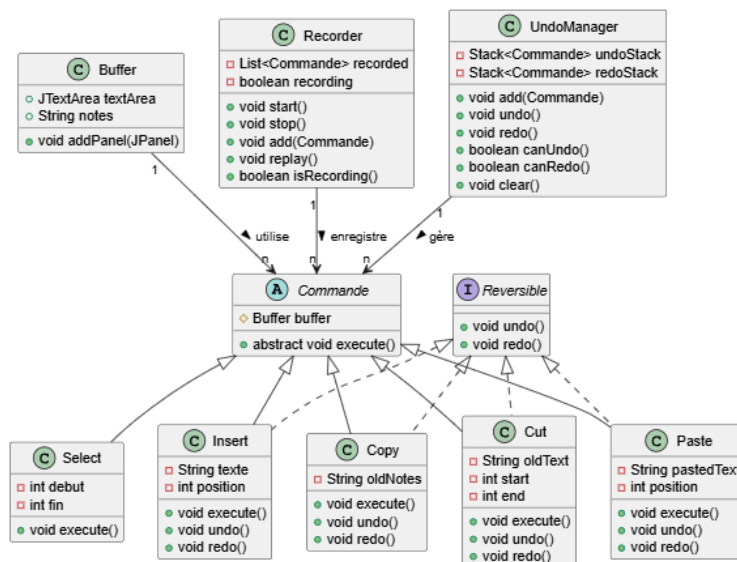


Buffer contient le texte et gère l'affichage.

Commande est une classe abstraite qui définit le lien avec le buffer.

Les classes concrètes Insert, Copy, Cut, Paste et Select héritent de Commande et réalisent chacune une action spécifique.

Diagramme de classe V2 :

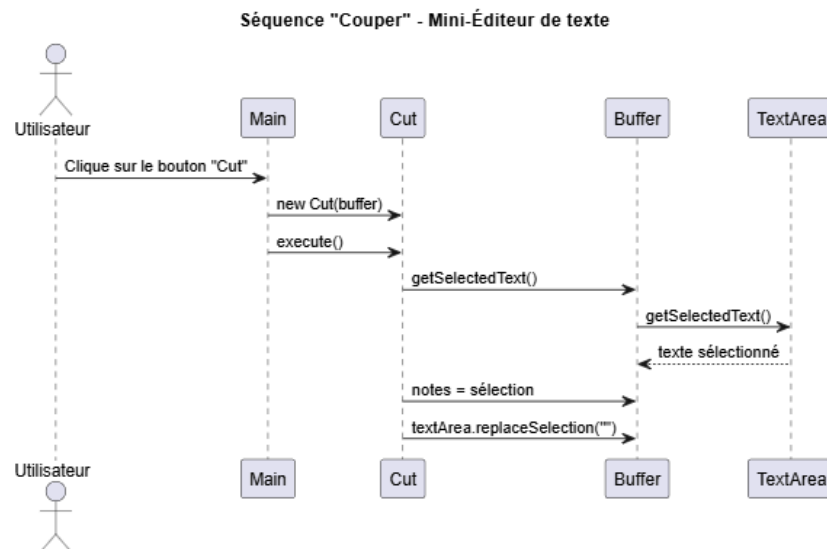


On ajoute les classes Recorder et UndoManager pour gérer l'enregistrement, la répétition et l'annulation.

Une interface Reversible définit les méthodes undo() et redo() pour les commandes modifiant le texte.

Diagrammes de séquence:

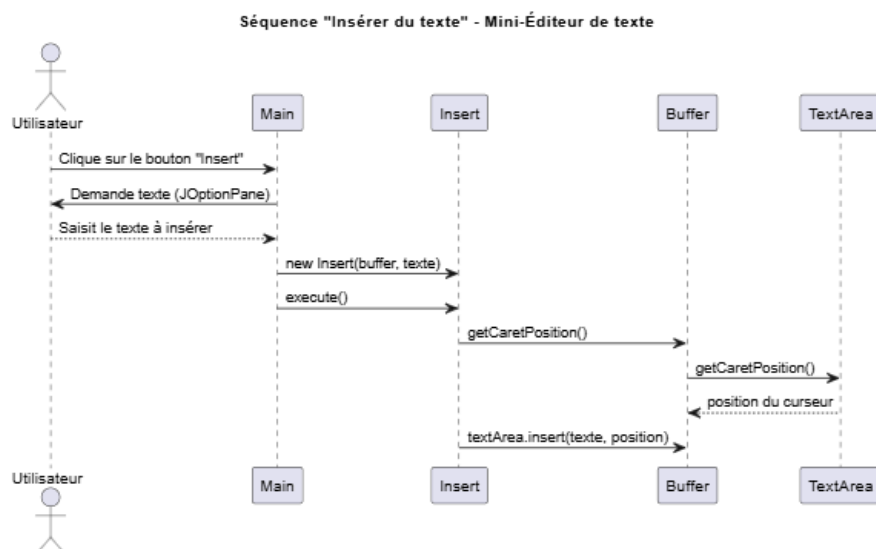
Séquence "Couper" :



L'utilisateur crée une instance de Cut et appelle execute().

La commande récupère le texte sélectionné, le stocke dans le presse-papiers du buffer, puis supprime la sélection.

Séquence "Insérer du texte" :



L'utilisateur saisit une chaîne et déclenche la commande Insert.

La méthode execute() insère le texte dans le buffer à la position du curseur.

Workspaces :

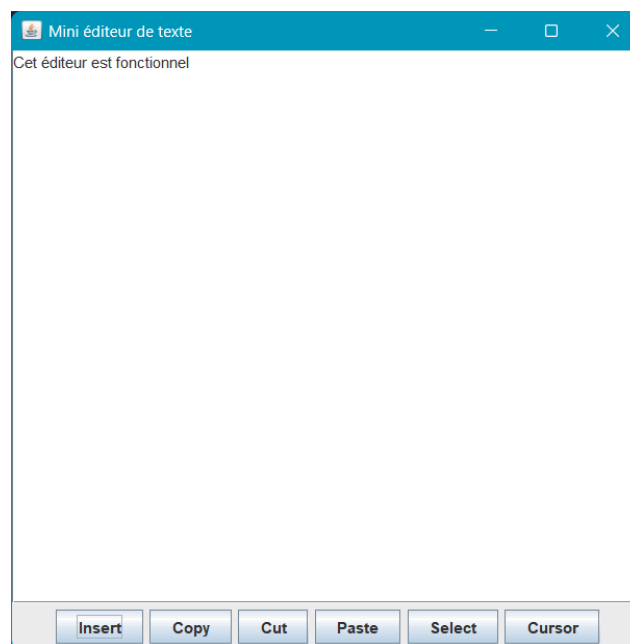
V1 :

```
project/
├── Commandes/
│   ├── Commande.java
│   ├── Insert.java
│   ├── Copy.java
│   ├── Cut.java
│   ├── Paste.java
│   └── Select.java
├── principales/
│   └── Buffer.java
└── Main.java
```

Main crée la fenêtre graphique avec les boutons.

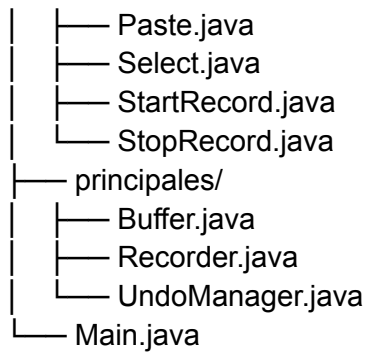
Buffer gère l'interface et le texte.

Chaque classe de Commandes réalise une opération sur le texte.



V2 :

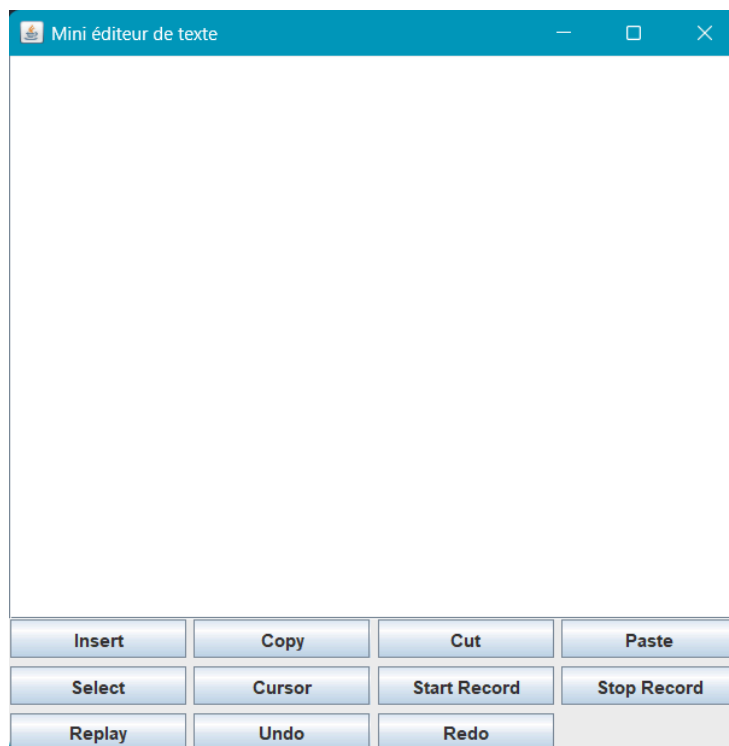
```
project/
├── Commandes/
│   ├── Commande.java
│   ├── Reversible.java
│   ├── Insert.java
│   ├── Copy.java
│   └── Cut.java
```



Recorder pour enregistrer et rejouer les commandes.

UndoManager pour la gestion d'annulation et de réexécution.

Interface Reversible pour les commandes annulables.



Pour enregistrer et rejouer des actions il faut :

- Appuyer sur Start Record pour commencer l'enregistrement.
- Effectuer plusieurs actions (ex : insérer, copier, coller...).
- Appuyer sur Stop Record pour arrêter.
- Appuyer sur Replay pour rejouer automatiquement la séquence enregistrée.

Sinon on peut appuyer sur Undo pour revenir en arrière et Redo pour rétablir une action annulée.

Conclusion :

Les diagrammes UML réalisés permettent de représenter la conception et les interactions entre les classes, facilitant la compréhension du fonctionnement global du programme.

La première version met en place les opérations de base sur le texte grâce à des commandes agissant sur le buffer.

La seconde version ajoute la gestion de l'historique et l'enregistrement des actions. Ce projet utilise une approche itérative et modulaire pour concevoir une application extensible.